

ContentFlow

Video Content Operations & Publishing Management System

Product Overview & Software Requirements Specification (SRS)

Prepared for the development team

Version 1.0 · September 26, 2026

1. Introduction

1.1 Purpose

This document specifies the requirements for ContentFlow, an internal web application that manages the full lifecycle of short-form video content — from planning and editing through to publishing on social platforms. It is intended to give the development team a single, unambiguous reference for scope, roles, workflows, data model, and acceptance criteria.

1.2 Problem Statement

Video editors finish videos and hand them off informally. Posting interns then have to track down which video to post, where the caption lives, and whether something has already gone out. This creates constant status-checking overhead: “Where is today's video?”, “Has this reel been edited?”, “Did you post yesterday's reel?” ContentFlow removes this overhead by making the content calendar the single source of truth for every video, caption, assignment, and status.

1.3 Core Workflow

Plan → Edit → Upload → Ready to Post → Download → Publish → Mark Posted

1.4 Guiding Principle

ContentFlow is a purpose-built content operations tool, not a generic project-management platform. It must stay lean: no CRM, accounting, chat, client management, or analytics in the first version. Every design decision should serve the single workflow above.

1.5 Definitions

Term	Meaning
Content item	A single scheduled piece of video/social content moving through the lifecycle.
Editor	Team member who produces and uploads the final video.
Posting intern	Team member who downloads the final asset and publishes it to the platform.
Workspace	A single-tenant container for users and content; architected to later support multiple brands.

Table 1.1 — Glossary of terms used throughout this document.

2. Overall Description

2.1 Product Perspective

ContentFlow is a standalone, internally-hosted web application with three role-based experiences (Admin, Editor, Posting Intern) sharing one calendar and one content database. It is built with a modern TypeScript/React stack, a relational database, and object storage for video assets.

2.2 User Classes and Characteristics

Role	Summary	Primary Goal
Admin / Manager	Full operational control: creates content, assigns work, manages the team, and monitors the whole pipeline.	"I can see everything happening with our content."
Video Editor	Sees only assigned content; uploads the final video and hands it off.	"I know exactly which videos I need to edit."
Posting Intern	Sees only assigned posting tasks; downloads, publishes, and confirms.	"I know exactly what I need to post today."

Table 2.1 — User classes.

2.3 Operating Environment

- Modern React-based frontend, responsive across desktop, laptop, tablet, and mobile.
- Relational database (e.g. PostgreSQL) for structured records.
- Object storage (e.g. S3-compatible) for video and thumbnail files, referenced by URL from the database.
- Server-side authentication and role-based authorization enforced on every request.
- Default timezone Asia/Kolkata, configurable in Settings.

2.4 Design & Implementation Constraints

- TypeScript across frontend and backend.
- All permission checks enforced server-side — never trust the client.
- No hardcoded credentials or secrets; use environment variables.
- Database schema should anticipate a future Workspace → Users → Content → Calendar hierarchy for multi-brand support, even though v1 runs a single workspace.

2.5 Assumptions and Dependencies

- Real publishing to social platforms is manual (performed by the posting intern outside the app); no social API integration in v1.
- Sample/placeholder videos may be used for demo data if real assets are unavailable.

3. User Roles & Permissions

3.1 Admin / Manager

Full access to every operation in the system.

- Create, edit, and delete content
- Assign and reassign editors and posting interns
- Change scheduled dates, times, captions, hashtags, and instructions
- Upload or replace videos on any content item
- Change status on any item
- View all content, the full calendar, team activity, and posting history
- Add, edit, and disable users; change roles

3.2 Video Editor

Scoped to assigned work only.

Can:

- View and open assigned content and the calendar
- Read the content brief and download reference files
- Upload the final video, replace it before posting, and add editor notes
- Submit content as “Ready to Post”
- See whether their submitted video has been posted

Cannot:

- Change posting ownership or the final posting date (unless explicitly permitted)
- Mark content as Posted
- Delete published content or manage users

3.3 Posting Intern

Deliberately the simplest experience in the product.

Can:

- View assigned posting tasks, today's content, and upcoming content
- Open content, preview and download the final video
- Copy caption and hashtags; view posting instructions
- Mark content as Posted, attach the post URL and actual posting time
- Report an issue

Cannot:

- Replace the final video or edit content strategy
- Change the assigned editor, delete content, or manage users

3.4 Permission Matrix

Action	Admin	Editor	Poster
Create / delete content	✓	—	—
Assign editor / poster	✓	—	—
Upload final video	✓	✓	—
Replace final video	✓	✓ (pre-posting)	—
Download final video	✓	✓	✓
Copy caption / hashtags	✓	✓	✓
Mark as Posted	✓	—	✓
Report an issue	✓	✓	✓
Manage users / roles	✓	—	—

Table 3.1 — Permission matrix. Enforce every row server-side.

4. Content Lifecycle

Every content item carries exactly one status at a time, driving both visibility and available actions.

Status	Meaning
PLANNED	Content created; editing has not started.
EDITING	Editor is actively working on the video.
READY_TO_POST	Final video uploaded; awaiting publication.
POSTED	Successfully published to the platform.
REVISION	Sent back to editing before it can be posted.
ISSUE	A blocking problem has been reported.

Table 4.1 — Status values. Suggested calendar colors: Planned = neutral grey, Editing = amber, Ready = green, Posted = blue, Revision/Issue = red.

4.1 Normal Flow

PLANNED → EDITING → READY_TO_POST → POSTED

4.2 Revision Flow

READY_TO_POST → REVISION → EDITING → READY_TO_POST

4.3 Key Transition Rules

- Uploading a final video automatically moves EDITING → READY_TO_POST.
- Marking as Posted automatically moves READY_TO_POST → POSTED and is irreversible from the Poster's view (Admin may reopen it).
- If status is already POSTED, hide the “Mark as Posted” action and show a read-only “Posted ✓” confirmation with who/when/URL instead.
- Overdue is a derived state, not a stored status: an EDITING item past its scheduled date, or a READY_TO_POST item past its scheduled time, should render as overdue.

5. Functional Requirements

Requirements are grouped by module and numbered for traceability (e.g. FR-CAL-3). Each should map to a ticket.

5.1 Authentication & Authorization

FR-AUTH-1 The system shall authenticate all users before granting access to any route.

FR-AUTH-2 The system shall enforce role (Admin / Editor / Poster) on the server for every read and write operation — never rely on frontend hiding alone.

FR-AUTH-3 Editors shall only retrieve content where they are the assigned editor; Posters shall only retrieve content where they are the assigned poster. Admins retrieve all content.

FR-AUTH-4 Disabled users shall be denied authentication immediately.

5.2 Content Creation & Management (Admin)

FR-CONTENT-1 Admin shall create content via a “+ Create Content” action capturing: title, description, content type, platform, scheduled date, scheduled time, editor, posting intern, caption, hashtags, instructions, and optionally a reference file, thumbnail, internal notes, and tags.

FR-CONTENT-2 Content type shall support: Reel, Short Video, Carousel, Static Post, Story, Other.

FR-CONTENT-3 New content shall default to status PLANNED.

FR-CONTENT-4 Admin shall be able to edit any field, reassign editor/poster, and change status on any content item.

FR-CONTENT-5 Admin shall be able to delete content; deletion shall be unavailable to Editor and Poster roles.

FR-CONTENT-6 Admin shall be able to duplicate a content item, copying title, description, caption, hashtags, instructions, and content type into a new record with a new ID, while excluding posted status, post URL, and activity history.

FR-CONTENT-7 Admin shall be able to drag a calendar item to a new date; the system shall confirm (“Move this content from [date] to [date]?”) before applying the change and shall log the change in activity history.

5.3 Calendar

FR-CAL-1 The system shall provide Month, Week, and Day calendar views, defaulting to Month.

FR-CAL-2 Each content item shall render on its scheduled date with a compact card showing: title, time, editor, poster, and a status indicator.

FR-CAL-3 Clicking a calendar card shall open the content detail view (page or modal).

FR-CAL-4 The calendar shall support search by title, editor, poster, platform, status, and date, and filtering by status, editor, poster, platform, content type, and date.

FR-CAL-5 The calendar shall remain responsive with at least several thousand content records (see Non-Functional Requirements).

5.4 Content Detail View

FR-DETAIL-1 The content detail view shall show title, scheduled date/time, and current status prominently at the top.

FR-DETAIL-2 It shall include a video preview with play, pause, seek, and fullscreen controls, plus file metadata (file name, size, uploaded at, uploaded by).

FR-DETAIL-3 It shall provide Download Video and, for authorized roles (Admin, assigned Editor), Replace Video actions.

FR-DETAIL-4 It shall display content information: platform, content type, scheduled date/time, editor, posting intern.

FR-DETAIL-5 It shall display the caption with a Copy Caption action that confirms “Caption copied ✓”, and the hashtags with a Copy Hashtags action.

FR-DETAIL-6 It shall display posting instructions as plain text.

FR-DETAIL-7 It shall display the full activity history for that content item in human-readable language (e.g. “Any uploaded final_video.mp4”), never raw database events.

5.5 Editor Workspace

FR-EDIT-1 The Editor dashboard shall show counts of Assigned, Editing, Ready to Post, Revision, and Posted items for that editor, plus a list of active work.

FR-EDIT-2 Editors shall be able to upload a final video (MP4, MOV, WEBM) with a visible progress indicator.

FR-EDIT-3 On successful upload, the system shall automatically: store the file, save the video URL, record the uploader and timestamp, change status to READY_TO_POST, write an activity log entry, and notify the assigned posting intern.

FR-EDIT-4 Editors shall be able to replace the uploaded video at any point before the content is Posted.

FR-EDIT-5 Editors shall be able to add optional notes to a content item.

5.6 Posting Intern Workspace

FR-POST-1 The Poster dashboard (“Today's Posting Queue”) shall list today's assigned posts with time, title, status, and Download / Open actions.

FR-POST-2 Posters shall be able to preview and download the final video directly from the detail page or the posting dashboard, without navigating a file-storage UI.

FR-POST-3 Posters shall be able to copy the caption and hashtags with one action each.

FR-POST-4 “Mark as Posted” shall open a confirmation dialog capturing: platform, posted-at time (pre-filled with current time, editable), post URL, and an optional note, with Cancel and Confirm Posted actions.

FR-POST-5 Confirming shall change status to POSTED and automatically save posted_by, posted_at, post_url, and platform, and write an activity log entry.

FR-POST-6 If a content item is already POSTED, the system shall hide “Mark as Posted” and instead show a read-only Posted confirmation with poster, time, and URL.

FR-POST-7 Posters shall be able to Report an Issue, selecting an issue type (video not downloading, wrong video, caption issue, editing problem, platform issue, cannot publish, other) with an optional comment; submitting sets status to ISSUE and notifies Admin and the assigned editor.

5.7 Admin Dashboard

FR-ADMIN-1 The Admin dashboard shall lead with “What needs my attention?”: counts of Ready to Post, Editing Overdue, Revisions, and Issues.

FR-ADMIN-2 It shall show top-level metrics: total content, and counts per status (Planned, Editing, Ready to Post, Posted).

FR-ADMIN-3 It shall list today's content (time, title, editor, poster, status) and an “Attention Required” section (overdue editing, overdue posting, revisions, issues).

FR-ADMIN-4 It shall support quick filters: Today, This Week, This Month, Overdue, Ready to Post, Posted, Issues.

5.8 Overdue Logic

FR-OVERDUE-1 An item shall be flagged Editing Overdue when its scheduled date has passed and its status is still EDITING.

FR-OVERDUE-2 An item shall be flagged Posting Overdue when its scheduled time has passed and its status is still READY_TO_POST.

FR-OVERDUE-3 Overdue flags shall be visually distinct but not alarming (e.g. a subtle warning color, not flashing or aggressive red).

5.9 Notifications

FR-NOTIF-1 The system shall notify the assigned posting intern when an editor uploads a final video and status becomes READY_TO_POST.

FR-NOTIF-2 The system shall notify the assigned editor when content moves to REVISION.

FR-NOTIF-3 The system shall notify Admin when a content item becomes overdue or an issue is reported.

FR-NOTIF-4 The header shall include a notification icon with an unread indicator; notifications include content date changes.

5.10 Activity Log

FR-LOG-1 The system shall record an activity entry for every meaningful state change: creation, status transitions, uploads, downloads, replacements, posting, URL additions, and date moves.

FR-LOG-2 Activity entries shall be rendered in plain, human-readable language, timestamped, and attributed to the acting user.

5.11 Team Management (Admin)

FR-TEAM-1 Admin shall have a Team page listing name, role, and status (Active/Disabled) for every user.

FR-TEAM-2 Admin shall be able to add a user, edit a user, disable a user, and change a user's role.

5.12 Settings (Admin)

FR-SET-1 Admin shall be able to configure workspace name, logo, default timezone (default Asia/Kolkata), default posting platform, and notification preferences.

5.13 Empty & Error States

FR-STATE-1 The system shall show a friendly empty state when an editor has no assigned content, when a poster has no posts today, and when Admin has no open issues.

FR-STATE-2 The system shall show a clear, specific message on upload failure, download failure, permission errors, and network errors, and shall allow the user to retry.

6. Non-Functional Requirements

6.1 Performance

- Lazy-load videos; use thumbnails in calendar and list views rather than full-resolution previews.
- Paginate large content lists; the calendar must stay usable with thousands of records.
- Show upload progress for video uploads; avoid unnecessary API calls; compress thumbnails.

6.2 Security

- All authorization enforced server-side; an Editor must not be able to perform Admin-only operations via direct API calls.
- Users only ever receive data they are authorized to see (scoped queries, not client-side filtering).
- Video assets served via signed/private URLs where appropriate; no public bucket listing.
- No secrets or credentials in source code; use environment variables.

6.3 Usability

- Clean, minimal, professional visual design with generous whitespace and clear status color-coding.
- UI copy in plain operational language (“Ready to Post”, “Download Video”, “Mark as Posted”) — never developer-facing phrasing.

6.4 Responsiveness

- Full application usable on desktop, laptop, and tablet.
- The posting workflow specifically must be fully usable on mobile: open today's content, download video, copy caption, copy hashtags, mark as posted — without needing a desktop.

6.5 Reliability & Data Integrity

- Duplicate posting must be structurally prevented once status is POSTED (see FR-POST-6).
- Every status transition must be atomic with its associated side effects (file save, activity log, notification).

7. Data Model

A relational database with the following primary tables. Video files themselves belong in object storage; the database stores only the reference URL.

7.1 users

Field	Notes
id	Primary key
name	
email	Unique
avatar	Optional
role	admin editor poster
status	active disabled
created_at / updated_at	

7.2 content

Field	Notes
id	Primary key
title, description	
content_type	reel short_video carousel static_post story other
platform	e.g. Instagram, YouTube, TikTok
scheduled_date, scheduled_time	
editor_id, poster_id	Foreign keys → users
caption, hashtags, instructions	
video_url, thumbnail_url	References to object storage
status	planned editing ready_to_post posted revision issue
post_url, posted_at, posted_by	Filled on posting
created_by, created_at, updated_at	

7.3 activity_logs

Field	Notes
id	Primary key
content_id	Foreign key → content

Field	Notes
user_id	Foreign key → users
action, description	Human-readable summary
metadata	Structured extra data, e.g. old/new values
created_at	

7.4 issues

Field	Notes
id	Primary key
content_id	Foreign key → content
reported_by	Foreign key → users
issue_type	See FR-POST-7
description	Optional comment
status	open resolved
resolved_by, resolved_at	
created_at	

7.5 Forward-Compatible Architecture

Structure the schema so a Workspace entity can be introduced above Users without a breaking migration:
 Workspace → Users → Content → Calendar. Run v1 with a single implicit workspace.

8. UI Structure & Navigation

8.1 Layout

Standard dashboard shell: top bar (logo, search, profile/notifications) and a left sidebar whose items change by role.

8.2 Navigation by Role

Admin	Editor	Poster
Dashboard	Dashboard	Dashboard
Calendar	Calendar	Calendar
Content	My Work	Posting Queue
Team	Activity	Activity
Activity		
Settings		

Table 8.1 — Sidebar navigation per role.

8.3 Reusable Components

Sidebar, Topbar, DashboardCard, Calendar, CalendarItem, ContentCard, ContentDetail, StatusBadge, VideoPlayer, VideoUploader, DownloadButton, CopyButton, PostingConfirmationModal, IssueModal, ActivityTimeline, NotificationDropdown, UserAvatar, FilterBar, SearchBar, EmptyState, LoadingState, ErrorState.

8.4 Mobile Posting View

A simplified single-column layout giving the posting intern, in order: title/time/status, video preview, Download Video, Caption with Copy, Hashtags with Copy, Instructions, and Mark as Posted — reachable without any desktop-only interaction.

9. Out of Scope for v1

The following are explicitly excluded from the MVP to keep the product focused. They may be revisited later (Section 10).

- AI video generation or AI-written captions
- Social media analytics or direct social-API auto-publishing
- CRM, billing, client management, or team chat/messaging
- Payroll, attendance, or complex reporting
- A visual automation/workflow builder
- Complex enterprise-style granular permissions beyond the three roles above

10. Future-Ready Extensions (Not Built Now)

The architecture should not preclude, but need not implement, the following:

- Multiple brands / multiple workspaces and calendars
- Client-facing accounts and approval workflows
- Social platform API integrations for direct/auto publishing
- Content performance analytics and reporting
- AI-assisted captions or content suggestions
- Additional cloud storage integrations

11. Acceptance Criteria

The build is considered complete when the following end-to-end scenario passes in full.

Test 1 — Admin creates content

- Admin creates a content item → it appears on the calendar with status PLANNED.

Test 2 — Editor uploads

- Editor logs in and sees the assigned item → uploads a video → it appears in the content detail view → status becomes READY_TO_POST.

Test 3 — Poster reviews

- Poster logs in and sees the item in the Posting Queue → can preview, download, copy caption, copy hashtags, and read instructions.

Test 4 — Poster publishes

- Poster publishes externally, returns, clicks Mark as Posted, adds the post URL → status becomes POSTED.

Test 5 — Admin verifies

- Admin sees the item as POSTED with uploader, poster, both timestamps, post URL, and full activity history visible.

11.1 System-Level Checklist

Beyond the scenario above, the following must be genuinely functional (not mocked): authentication, role-based access, database persistence, content creation, assignment, calendar (month/week/day), video upload/storage/preview/download, caption and hashtag copying, all status transitions, the posting confirmation flow, activity logs, notifications, search, filters, and responsive layouts on mobile.

12. Recommended Build Order

Phase	Scope
1	Authentication + roles
2	Database + content model
3	Admin content creation
4	Calendar (month / week / day)
5	Editor workspace
6	Video upload & storage
7	Posting intern workspace
8	Download + copy actions
9	Mark as Posted workflow
10	Activity logs
11	Notifications
12	Overdue logic
13	Search / filters
14	Responsive / mobile optimization
15	Testing and polish

Table 12.1 — Build phases, in order. Each phase should ship the workflow it supports end-to-end before moving on.

NOW MAKE SURE THIS GMAIL YHEY CAN LOGIN THROUGH GOOGLE ALSO IF THEY DIRECTLY CLICK CONTINUE WITH GOOGLE BUT ONLY THIS MAILS

Admin : quickuppsotech.cmo@gmail.com
snehalpawar12014@gmail.com

MANAGER : DMM : THIS MAIL I WILL GIVE YOU IN SOMETIMES

USERS:
GRAPHIC DESIGNER : Devyani Ankush Bhoye

qs.graphicdesingner@gamil.com

Rutuja Ganesh Pawar

qsgraphicdesigner2@gmail.com

Swapnil Nawadkar

quickupp.graphicdesigns@gmail.com

VIDEO EDITOR :

Ubaid Maner

dmquickuppsotech@gmail.com

Rahul sanjay mahajan

qs.photography0079@gmail.com

INTERNS :

Pratiksha Magatrao

qsdmintern01@gmail.com

qsdmintern4@gmail.com

tanisha suresh bangde

Gayatri Ratnakar Sitafale

qsintern009@gmail.com

Kiran B Arote

qsdmintern03@gmail.com